QR MENU SAAS

Full Project Blueprint

A subscription-based digital menu platform for restaurants, cafés, bakeries and food businesses

Includes
Product requirements • customer and admin dashboards • customizable templates • QR menu publishing • PostgreSQL data model • Django API design • recurring subscriptions • autopay/mandates • payment webhooks • security • deployment • testing • launch roadmap

Recommended stack

Frontend: React + Vite + TypeScript • **Backend:** Django + Django REST Framework • **Database:** PostgreSQL • **Payments:** Razorpay Subscriptions or another gateway supporting recurring mandates • **Hosting:** managed application hosting + object storage

Version 1.0 • Prepared 22 October 2026 • Planning document, not production code

1. Executive summary

Build one multi-tenant SaaS application that lets each restaurant create an account, choose a design template, customize its digital menu, publish a permanent public menu URL, and download a QR code. Customers scan the QR code and view the live menu in a mobile browser. Restaurant owners pay a recurring subscription for access to paid features.

Core principle: one platform, many isolated restaurant accounts. Do not build a separate codebase or website for every restaurant. The restaurant's menu content, theme configuration, QR URL and subscription status are stored as data.

Goals for the first release

- Restaurant sign-up, login, email verification and account recovery.
- Restaurant profile, menu categories, menu items, photos, prices and availability.
- Several responsive templates with editable colors, fonts, logo, header, background, card layout and category order.
- Live preview and publish/unpublish controls.
- Stable public menu URL and downloadable QR code that does not change when menu content changes.
- Monthly and annual subscription plans, hosted checkout, autopay/recurring mandate consent, payment status and webhook-driven renewals.
- Platform admin tools to manage restaurants, templates, plans, billing status and support.

Out of scope for MVP

- Full POS, inventory, kitchen display, delivery fleet, table ordering, tax invoicing and multi-branch franchise controls.
- Customer accounts or a native mobile app; the public menu should work without installing an app.
- Building a custom payment processor or storing card/bank credentials.

2. User roles and journeys

Roles

Role	Permissions
Visitor / customer	Open a public menu, browse categories and items, view prices and offers, use optional contact/order links.
Restaurant owner	Manage their restaurant profile, menu, theme, images, publication, QR code and subscription.
Restaurant staff (optional MVP+)	Limited menu editing permissions; cannot manage billing or transfer ownership.
Platform administrator	Manage all tenants, templates, plans, subscriptions, support and platform settings.

Restaurant onboarding journey

- Create account → verify email → create restaurant profile and unique slug.
- Choose a plan → view billing terms → complete hosted checkout and explicitly authorize recurring payments if offered.
- Choose a template → customize settings → add categories and items → preview on mobile.
- Publish menu → generate QR code → download print-ready PNG/SVG/PDF (choose supported formats).
- Edit a price or mark an item sold out → save/publish → existing QR code shows the updated menu.
- Manage subscription, view next billing date, update payment method, cancel autopay or renew according to gateway support.

3. Functional requirements

3.1 Restaurant dashboard

- Dashboard summary: menu status, scans/views, plan, payment state and renewal date.
- Profile: business name, address, phone, opening hours, logo, cover image, social links and supported languages.
- Menu editor: create categories, reorder categories, add/edit/delete items, item description, price, image, veg/non-veg labels, tags, availability and featured status.
- Bulk-friendly editing is a later enhancement; MVP can use simple forms with image previews.
- Autosave or explicit save with unsaved-change warnings; publishing should be a deliberate action.
- Theme editor: template selection, color palette, font pairing, header layout, background image/color, item-card style, image aspect ratio and banner.
- QR page: preview, copy public URL, download QR, optionally generate table-specific codes in a later version.
- Billing page: current plan, next charge date where provided, payment history, invoices/receipts where supported, change plan, update payment method and cancel subscription.

3.2 Public menu

- Fast mobile-first page; no login required for customers.
- Display restaurant branding, opening hours, menu categories and items.
- Category navigation, optional search, item availability and clear prices.
- Accessible contrast, readable type, optimized images and graceful loading/error states.
- SEO/share metadata for public menu pages, but do not expose private owner or billing information.
- If subscription is overdue or canceled, apply a clearly defined grace-period and read-only/suspended-menu policy; communicate this before purchase.

3.3 Platform admin

- Search/filter restaurants and inspect status, plan and recent billing events.
- Create, update, enable/disable templates and default theme settings.
- Configure plans and feature limits; never hard-code pricing in multiple frontend locations.
- View payment webhook logs and retry safe internal processing when appropriate.
- Suspend/restore accounts with audit trail; avoid deleting customer menu data automatically.
- Track support tickets and operational incidents in a future release.

4. Subscription billing and autopay design

This section is central to the project. Use a payment gateway's official subscription/recurring-payment product. For an India-first launch, evaluate Razorpay Subscriptions and compare current support, pricing, mandate flows, supported payment methods, settlement and cancellation behavior against alternatives. Gateway capabilities and regulatory requirements can change, so verify the live documentation and your merchant account eligibility before implementation.

Autopay is not a hidden or automatic debit. The restaurant owner must see the plan amount, billing frequency, applicable taxes/fees, recurring-payment terms, mandate/payment method details and cancellation process, then explicitly authorize the recurring mandate through the gateway's supported flow.

4.1 Proposed plan configuration

Plan	Illustrative price	Possible limits/features
Starter	■299/month	1 restaurant, 1 published menu, basic templates, QR download.
Professional	■599/month	Premium templates, full theme editor, offers, analytics, multiple languages.
Business	■999/month	Higher limits, multi-branch support when implemented, advanced analytics and branding controls.
Annual	Set discount, e.g. 2 months free	Annual recurring charge if supported and clearly disclosed.

Prices are example launch hypotheses, not market research. Keep plan price, currency, interval, trial days, feature flags and limits in database/configuration. Show final payable amount and tax treatment clearly at checkout.

4.2 Recommended payment lifecycle

- 1. Owner selects a plan and billing interval.
- 2. Backend creates a pending subscription/order using the gateway's server-side API; secret keys never go to the browser.
- 3. Frontend redirects to or opens the gateway's hosted checkout/mandate authorization experience.
- 4. Owner completes payment and explicitly authorizes recurring autopay if the chosen method supports it.
- 5. Gateway sends signed webhook events to the backend. Backend verifies the signature against the raw request body and records the event idempotently.
- 6. Activate paid features only after authoritative gateway confirmation of the required successful payment/active subscription state; do not trust the browser success redirect alone.
- 7. On each renewal, rely on verified webhook events to update payment and subscription state.
- 8. On failed renewal, mark payment past_due, notify the owner, follow gateway retry behavior, and apply a disclosed grace period.
- 9. On cancellation, stop future renewals according to gateway rules, retain access until the paid-through date if policy allows, and show cancellation confirmation.
- 10. On plan change, define proration/effective date behavior and test upgrades, downgrades and failed payments.

4.3 Subscription state model

State	Meaning / application behavior
pending	Checkout or mandate authorization not completed; paid features remain locked.
trialing	Optional trial active; disclose when billing starts and how to cancel.
active	Subscription and required payment state are confirmed; grant plan entitlements.
past_due	Renewal failed or payment is overdue; notify, retry as allowed, start disclosed grace period.
cancel_at_period_end	Cancellation scheduled; access remains until paid-through date if policy states so.
canceled	No future renewal; apply post-term access policy.
paused / suspended	Use only if gateway and business rules support it; document recovery path.

4.4 Autopay, mandate and compliance checklist

- Use gateway-hosted collection/mandate authorization wherever possible; never store raw card numbers, CVV, UPI PIN, net-banking credentials or mandate secrets.
- Clearly disclose recurring amount, interval, first debit timing, any trial, applicable taxes, cancellation method and what happens after failed payment.
- Provide an easy way to cancel/stop future renewals and a clear confirmation/record of the request.
- Handle mandate authentication, pre-debit notifications, debit limits, cooling-off or other rules according to the current gateway flow and applicable RBI/NPCI requirements for the chosen payment method. Do not implement these from assumptions; confirm with the gateway.
- Verify webhooks, use idempotency keys where supported, store gateway customer/subscription/payment IDs, and reconcile status if webhook delivery is delayed.
- Send billing notifications for successful payments, failed renewals, expiring cards/mandates where supported, cancellations and plan changes.
- Publish terms of service, privacy policy, refund/cancellation policy, support contact and invoice/receipt process.
- Test sandbox and production separately; protect live keys, restrict dashboard access, rotate credentials and audit administrative changes.

4.5 Webhook processing rules

- Verify the gateway signature before processing any event.
- Persist each event ID in a webhook_events table with a unique constraint; duplicate deliveries must not duplicate charges or grant extra entitlement.
- Respond quickly after durable receipt; process heavier tasks asynchronously if needed.
- Expect out-of-order events. Fetch the current subscription/payment state from the gateway when events conflict or are ambiguous.
- Never trigger a debit merely because a webhook was received. Charges should follow the gateway's authorized subscription/mandate schedule.
- Log event type, gateway reference, processing outcome and timestamp without logging secrets or unnecessary personal data.

5. Technical architecture

Recommended MVP stack

Layer	Recommendation	Why
Frontend	React + Vite + TypeScript + Tailwind CSS	Reusable dashboard, template editor and responsive public menus.
Backend	Django + Django REST Framework	Authentication, admin, permissions, REST APIs and data validation.
Database	PostgreSQL	Relational menu and billing data, constraints and reliable transactions.
Images	S3-compatible object storage + CDN	Avoid storing large images directly in database rows.
QR	Python QR library	Generate QR pointing to permanent public menu URL.
Payments	Gateway subscription API + hosted checkout	Recurring mandate, payment processing and webhook lifecycle.
Email	Transactional email provider	Verification, reset, billing alerts and receipts.
Deployment	Managed app/container hosting + managed PostgreSQL	Simpler operations than self-managing every service initially.

High-level architecture

Browser (owner/customer) → HTTPS reverse proxy/CDN → React frontend and Django REST API → PostgreSQL. Uploaded images go to object storage. Django calls the payment gateway server-to-server. Gateway webhooks return to a dedicated verified endpoint. Email and analytics events can be handled by a background worker as traffic grows.

6. Data model

Use UUID primary keys where practical. Every tenant-owned row must be linked to a restaurant and every query must enforce that the current user has access to that restaurant. A unique slug creates the public URL. Store money as integer minor units (paise) or a properly configured decimal type; do not use floating-point for money.

Table	Important fields
users	id, email, password_hash (managed by Django), is_active, email_verified_at, created_at
restaurants	id, owner_user_id, name, slug UNIQUE, logo_key, cover_key, contact_json, timezone, published_at, status
restaurant_members	id, restaurant_id, user_id, role, status; unique restaurant/user pair
plans	id, code UNIQUE, name, price_minor, currency, billing_interval, feature_config JSONB, is_active
subscriptions	id, restaurant_id, plan_id, gateway, gateway_customer_id, gateway_subscription_id UNIQUE, status, current_period_start/end, cancel_at_period_end
payments	id, subscription_id, gateway_payment_id UNIQUE, amount_minor, currency, status, paid_at, receipt_url/reference
webhook_events	id, gateway_event_id UNIQUE, event_type, received_at, processed_at, processing_status, safe_payload
templates	id, name, code UNIQUE, preview_image_key, default_config JSONB, version, is_active
menus	id, restaurant_id, title, template_id, theme_config JSONB, is_published, published_at
categories	id, restaurant_id, menu_id, name, sort_order, is_active
menu_items	id, restaurant_id, category_id, name, description, price_minor, image_key, dietary_label, is_available, sort_order
qr_codes	id, restaurant_id, menu_id, public_url, format, created_at
menu_events	id, restaurant_id, menu_id, event_type, occurred_at, coarse_metadata; avoid unnecessary tracking

Use JSONB for flexible theme settings, not for every business entity. Keep categories, menu items, subscriptions and payments in relational tables so they can be validated and queried. Add indexes to foreign keys, slugs, gateway IDs, status fields and common admin filters.

7. Example theme configuration

```
{
"template": "modern-cafe",
"version": 1,
"brand": {
"primary": "#5B4636",
"background": "#FFF9F0",
"text": "#25211E",
"fontHeading": "Playfair Display",
"fontBody": "Inter"
},
"header": {
"showLogo": true,
"coverImage": "restaurants/abc/cover.webp",
"title": "Example Cafe",
"subtitle": "Coffee • Brunch • Desserts",
"alignment": "center"
},
"menu": {
"cardStyle": "rounded",
"imageRatio": "4:3",
"showDescriptions": true,
"showDietaryLabels": true
}
}
```

The frontend renderer maps approved configuration values to predefined components. Do not accept arbitrary HTML or JavaScript from restaurant owners. Validate uploaded file types, image sizes, color/font options and configuration schema.

8. API endpoint plan

Use versioned REST endpoints, for example /api/v1/. Require authentication for owner/admin routes and enforce restaurant membership/role checks server-side. Public menu endpoints should return only published, customer-safe data.

Method / endpoint	Purpose
POST /auth/register	Create user account.
POST /auth/verify-email	Verify email using a time-limited token.
POST /auth/login	Create session/token using a secure authentication strategy.
POST /auth/password-reset	Start password reset flow.
GET/POST /restaurants	List accessible restaurants or create a restaurant.
GET/PATCH /restaurants/{id}	Read/update authorized restaurant profile.
GET/POST /menus	List/create menus for a restaurant.
PATCH /menus/{id}/theme	Validate and save theme settings.
POST /menus/{id}/publish	Publish menu after validation.
POST /menus/{id}/unpublish	Remove menu from public view.
POST /categories / PATCH /categories/{id}	Create/update category and order.
POST /items / PATCH /items/{id} / DELETE /items/{id}	Manage menu items with tenant checks.
GET /public/menu/{slug}	Serve published customer-facing menu.
POST /billing/checkout	Create gateway subscription/checkout session server-side.
GET /billing/subscription	Return current plan and safe subscription status.
POST /billing/change-plan	Request plan change with defined proration rules.
POST /billing/cancel	Request cancellation through gateway API.
POST /webhooks/payment-gateway	Verify signature, persist event and update billing state idempotently.
GET /admin/restaurants	Platform admin search/list.
GET /admin/billing-events	Platform admin payment event troubleshooting.

API response rules

- Use consistent validation errors and HTTP status codes.
- Paginate large lists and filter/sort on the server.
- Rate-limit login, password reset, public analytics and webhook-adjacent endpoints as appropriate.
- Never expose gateway secret keys, internal webhook payloads, private storage keys or other restaurants' IDs/data unnecessarily.
- Use signed upload URLs for direct-to-object-storage uploads, with size/type restrictions and expiry.

9. Billing implementation flow (pseudocode)

```
# Illustrative pseudocode only — adapt to the chosen gateway's current SDK/API.

def create_subscription_checkout(user, restaurant, plan):
require_owner_or_billing_admin(user, restaurant)
require_plan_is_active(plan)
# Create pending local record first; no entitlement yet.
local_sub = Subscription.objects.create(
restaurant=restaurant,
plan=plan,
status="pending",
gateway="chosen_gateway",
)
# Call gateway server-side using secret credentials.
gateway_sub = gateway_client.create_subscription(
plan_reference=plan.gateway_plan_reference,
customer_reference=get_or_create_gateway_customer(restaurant),
callback_url=APP_URL + "/billing/return",
# Exact mandate parameters depend on the gateway.
)
local_sub.gateway_subscription_id = gateway_sub.id
local_sub.save(update_fields=["gateway_subscription_id"])
return gateway_sub.hosted_checkout_url

def gateway_webhook(request):
verify_gateway_signature(request.body, request.headers)
event = parse_gateway_event(request.body)
# Unique event ID makes retries idempotent.
saved_event, created = save_event_if_new(event)
if not created:
return HTTP_200_OK
# Confirm authoritative payment/subscription state with gateway if needed.
current_state = fetch_or_validate_gateway_state(event)
update_local_billing_state(current_state)
recalculate_restaurant_entitlements()
return HTTP_200_OK
```

Do not copy the pseudocode as a drop-in integration. Gateway API fields, mandate parameters, signatures, retry behavior and event names differ and must follow current official documentation.

10. Security, privacy and reliability

Authentication and access control

- Use Django's password hashing and secure authentication practices; require HTTPS.
- Enforce tenant isolation on every read/write operation, not only in the UI.
- Use role-based permissions for owner, staff and platform admin.
- Protect sessions/cookies with secure settings, CSRF protections where applicable, and rate limits on authentication.
- Use least-privilege production credentials and keep secrets in environment/secret management, not source control.

Uploads and public pages

- Validate file extension, MIME type and decoded file format; enforce size and dimensions.
- Generate safe object names; do not trust user-supplied paths.
- Optimize and resize images; use CDN/cache headers for published public assets.
- Escape text and validate rich text; prohibit user-provided scripts and arbitrary HTML.
- Provide privacy notice for analytics; collect only what is needed and define retention.

Backups and operations

- Automated database backups and a tested restore procedure.
- Error monitoring, structured logs and alerts for failed jobs/webhooks.
- Audit log for plan changes, cancellation, account suspension and admin actions.
- Scheduled reconciliation of gateway subscription/payment status to recover from missed webhooks.
- Document incident response, support ownership and production rollback process.

11. Deployment architecture and cost planning

Start with a small managed deployment; scale when actual traffic and image storage justify it. These are rough planning ranges, not live vendor quotes.

Cost area	Early-stage planning range	Notes
App hosting	■1,000–■4,000/month	Depends on provider, resources and traffic.
Managed PostgreSQL	■1,000–■4,000/month	Backups, storage and compute affect cost.
Image/object storage + CDN	■200–■1,500/month	Depends on uploads, bandwidth and retention.
Email/domain/monitoring	■300–■1,500/month	Plan, email volume and services vary.
Payment gateway	Variable	Transaction and subscription fees depend on gateway and contract.
Estimated early infrastructure subtotal	■2,500–■11,000/month	Excludes development, taxes, payment fees and support.

Production deployment checklist

- Separate development, staging/sandbox and production environments.
- Configure domain and HTTPS; set allowed hosts, CORS, CSRF origins and secure cookies.
- Provision managed PostgreSQL with private access, backups and restricted credentials.
- Configure object storage bucket permissions and CDN.
- Set gateway keys and webhook secret in secret storage; register the production webhook URL.

- Run migrations, create admin user securely, configure email, logs and alerts.
- Set up automated tests and a deployment rollback plan.

12. Testing checklist

Area	Required tests
Tenant isolation	Restaurant A cannot read/edit Restaurant B's profile, menus, images, billing or analytics.
Menu editing	Create/edit/delete item; category ordering; invalid prices; unavailable items; publish/unpublish.
Theme editor	All templates render on mobile/desktop; invalid config rejected; uploaded backgrounds optimized.
QR	Correct permanent URL, downloadable file scans correctly, menu changes appear without new QR.
Payments	Successful checkout; user closes checkout; failed payment; duplicate webhook; delayed/out-of-order webhook; cancellation; renewal; plan change; refund if supported.
Autopay	Explicit mandate consent; recurring terms shown; supported mandate flow; failed debit and retry; cancellation/stop mandate; customer notifications.
Security	Unauthorized API calls, brute force/rate limits, upload validation, XSS attempts, secret leakage and admin access.
Operations	Database backup/restore, email delivery, webhook replay/reconciliation, monitoring and deployment rollback.

13. Suggested development roadmap

A realistic first version depends on developer experience, design readiness and payment-gateway onboarding. For a small team, plan in milestones rather than promising a fixed launch date before the payment flow is tested.

Milestone	Work	Definition of done
M1 — Foundation	Repo, environments, auth, restaurant model, tenant permissions, PostgreSQL.	A user can securely create and access only their restaurant.
M2 — Menu engine	Categories/items, image upload, public menu route, publish state.	Public menu renders correctly from database content.
M3 — Templates	2–4 responsive templates, theme JSON schema, editor and preview.	Owner can customize and publish without code changes.
M4 — QR	Stable slug, QR generation, download and scan tests.	Printed QR opens the correct live menu.
M5 — Billing	Plans, hosted checkout, recurring mandate, webhooks, billing page.	Sandbox subscription, renewal, failure and cancellation scenarios pass.
M6 — Admin & QA	Admin screens, logs, alerts, backups, security and mobile QA.	Release checklist complete and pilot users onboarded.
M7 — Pilot launch	Onboard 3–5 businesses, observe usage, fix issues.	Owners can update menus and understand billing/cancellation.

14. MVP launch acceptance criteria

- A restaurant owner can register, verify the account and create a restaurant profile.
- Owner can choose a template, edit theme properties, add categories and items, and preview the result.
- Published menu has a stable URL and QR code; menu changes do not require a new QR code.
- Customer menu works on current mobile browsers and does not require an account.
- Owner can select a plan and complete the gateway's recurring authorization flow with clear consent.
- Subscription access is changed only from verified server-side payment/subscription state.
- Duplicate webhooks are safe; failed renewals, cancellations and plan changes are handled and tested.
- Owner can see billing state and cancel future renewal through a documented path.
- Platform admin can inspect restaurants and safely troubleshoot billing events.
- Tenant isolation, backup/restore and production secret management are tested.

15. Key product decisions to settle before coding

- Will customers only view menus, or can they place orders? Recommendation: view-only MVP.
- Will a subscription cover one restaurant or multiple branches? Recommendation: one restaurant per subscription initially.
- What happens to a menu after failed renewal? Define grace period, read-only state and customer-facing message before launch.
- Which templates and customization settings are available on each plan?
- Which gateway and recurring payment methods are approved for your merchant account?
- Will you provide free trials, annual discounts, refunds and prorated plan changes?
- Who handles customer support, failed payments, cancellation disputes and privacy requests?

16. Official documentation to verify before implementation

Use current official docs for the gateway and framework versions selected at build time. This PDF deliberately avoids hard-coding unverified payment API details because gateway products, mandate rules and webhook names can change.

- Django: https://docs.djangoproject.com/
- Django REST Framework: https://www.django-rest-framework.org/
- PostgreSQL: https://www.postgresql.org/docs/
- Razorpay Subscriptions: https://razorpay.com/docs/payments/subscriptions/
- Razorpay Webhooks: https://razorpay.com/docs/webhooks/
- React: https://react.dev/
- Vite: https://vite.dev/guide/

Payment/compliance note: before launch in India, confirm the current RBI/NPCI requirements that apply to the specific recurring payment method and gateway mandate flow. This document is a technical planning guide, not legal, tax or payment compliance advice.

End of project blueprint • Next practical step: turn this blueprint into a screen-by-screen UI specification and a Django/PostgreSQL implementation backlog.